

# Sensing-Project 全栈化重构平台 — 技术文档

版本: v2.0 | 日期: 2026-05-12 | 适用于 sqc v0.1.0

本文档按照 **Gao, Rol, Touzard, Wang 2021** (PRX Quantum 2, 040202) 所提出的 cQED 六层全栈架构组织。每一章既给出物理动机, 又详尽介绍代码模块、扩展接口与开发规范。

## 目录

- 1. 平台概览 — 项目定位、三大科研主线、关键特性
- 2. 物理背景 — Transmon、cavity、耦合、传感原理、参数范围
- 3. 全栈架构 — 六层模型、依赖规则、目录树、与 Gao 2021 对应表
- 4. 模块详解 — 逐层逐文件的细致介绍、物理含义、关键 API
  - 4.1 devices/ — 物理器件层
  - 4.2 hardware/ — 控制电子学层
  - 4.3 control/ — 控制脉冲层
  - 4.4 simulation/ — 仿真层
  - 4.5 experiments/ — 实验协议层
  - 4.6 reconstruction/ — 波形重建层
  - 4.7 calibration/ — 标定层
  - 4.8 workflows/ — 顶层科研流程
- 5. 数据流 — 端到端管道、跨层调用顺序
- 6. 快速开始 — 安装、最小示例、向后兼容
- 7. 扩展指南 — 添加协议、算法、失真模型、qubit 种类的完整流程
- 8. API 参考 — 全部公共类、关键函数签名、Cheat Sheet
- 9. 测试与回归 — 测试组织、baseline 协议、CI 守则
- 10. 全局配置 — `sqc.config` 设计与使用
- 11. 设计原则与约定 — 单位、命名、不变量、与 Gao 论文的差异
- 12. 附录: Gao 2021 公式对应表 — 每个模块到论文公式/章节的指针
- 参考文献

## 1. 平台概览

### 1.1 项目定位

Sensing-Project 是一个**研究型量子传感仿真框架**, 以 flux-tunable Transmon 超导量子比特为物理对象, 研究**时变磁通信号的传感、重建、标定与反演**。

平台将超导量子计算机的控制栈从"单文件脚本"升级为**六层全栈架构**, 每一层对应真实 cQED (circuit QED) 实验中的物理组件或控制逻辑。这一组织方式直接来自 Gao et al. 2021 (PRX Quantum 2, 040202), 是当前 cQED 工程实践的事实标准。

### 1.2 三大科研主线

| # | 主线   | 物理目标                                      | sqc/ 归属             | Gao 2021 章节 |
|---|------|-------------------------------------------|---------------------|-------------|
| 1 | 波形重建 | 从 Ramsey / Cryoscope / 瞬态测量恢复片上 $\Phi(t)$ | sqc/reconstruction/ | §V.E        |

| # | 主线       | 物理目标                                   | sqc/ 归属                             | Gao 2021 章节  |
|---|----------|----------------------------------------|-------------------------------------|--------------|
| 2 | qubit 标定 | 标定 $f_{01}$ , $f(\Phi)$ , 灵敏度 $\kappa$ | sqc/calibration/                    | §V.A–V.B     |
| 3 | 波形预失真    | 建模 AWG→芯片传递函数, 设计补偿滤波器                 | sqc/hardware/ +<br>sqc/calibration/ | §V.E + §IV.B |

1.3 关键特性

- 纯 Python + QuTiP: 基于 QuTiP 的 `mesolve()` 进行全密度矩阵时间演化
- 自然单位制:  $\hbar=1$ , 频率/能量单位 rad·GHz, 时间单位 ns, 磁通单位  $\Phi_0$
- 不可变参数对象: QubitSpec 使用 `@dataclass(frozen=True)`, 杜绝"物理参数被实验状态污染"
- 无副作用 Hamiltonian 构造: `HamiltonianBuilder.build()` 纯函数, 输入 device + flux + pulse, 输出 `H_list`
- 永久向后兼容: `src/` 目录永久保留, `src_mirror/` 提供从 `sqc/` 重导出的 facade
- 全局配置中心化: 所有时间网格、AWG 参数、qubit 默认值从 `sqc.config.CONFIG` 单一来源推导 (v2.0 引入)
- 物理回归测试: 7 个 baseline pickle 锁定数值, `rtol=1e-6`, `atol=1e-9`

1.4 技术栈

| 层级    | 依赖                                                                                                 |
|-------|----------------------------------------------------------------------------------------------------|
| 数值底层  | NumPy $\geq$ 1.24, SciPy $\geq$ 1.11                                                               |
| 量子动力学 | QuTiP $\geq$ 5.0                                                                                   |
| 可视化   | Matplotlib $\geq$ 3.7, Gradio $\geq$ 4.0                                                           |
| 数据结构  | Python <code>dataclasses</code> (标准库)                                                              |
| 测试    | pytest $\geq$ 7.4, pytest-xdist, pytest-cov                                                        |
| 语言版本  | Python $\geq$ 3.10 (需要 <code>match-case</code> 、 <code>from __future__ import annotations</code> ) |

2. 物理背景

2.1 Transmon 量子比特

Transmon 是一种基于约瑟夫森结的超导量子比特, 其哈密顿量为 (Koch 2007; Gao 2021 §II.B Eq. 13):

$$H = 4 \cdot EC \cdot n^2 - EJ(\Phi) \cdot \cos(\phi)$$

其中:

- $EC = e^2/(2C\Sigma)$ : 充电能 (单电子库仑能)
- $EJ(\Phi) = EJ_0|\cos(\pi\cdot\Phi/\Phi_0)|$ : 有效约瑟夫森能 (SQUID 调制, Gao Eq. 20)
- $n, \phi$ : Cooper 对数和超导相位算符

在  $EJ/EC \gg 1$  的 Transmon 区域, 哈密顿量可展开为 Duffing 振子形式 (Gao Eq. 15–18):

$$H \approx \omega_T \cdot (n + \tfrac{1}{2}) + (\alpha/2) \cdot (n^2 - n)$$

- $\omega_{\mathbf{T}(\Phi)} = \sqrt{(8 \cdot EJ(\Phi) \cdot EC) - EC}$ :  $|0\rangle \rightarrow |1\rangle$  跃迁频率 (rad-GHz) (Gao Eq. 18)
- $\alpha = -EC$ : 非谐性 (anharmonicity), 保证二能级子空间可寻址

## 2.2 磁通传感原理

外部磁通  $\Phi(t)$  改变  $EJ(\Phi)$ , 进而改变  $\omega_{\mathbf{T}(\Phi)}$ 。通过 Ramsey 干涉测量 qubit 的相干相位累积:

$$\phi(\tau) = \int_0^\tau \Delta\omega(t') dt' = \int_0^\tau \kappa \cdot \Phi(t') dt'$$

其中  $\kappa = d\omega_{\mathbf{T}}/d\Phi$  是频率-磁通灵敏度 (flux sensitivity)。Ramsey 序列  $(\pi/2)-\tau-(\pi/2)$  之后测量  $p_e(\tau)$ , 从  $p_e(\tau) = \frac{1}{2}(1 - \cos[\phi(\tau)])$  反演  $\phi(\tau)$ , 进而恢复  $\Phi(t)$  (详见 §4.6)。

## 2.3 谐振腔与色散读出

Resonator (cavity) 与 qubit 耦合时, 由色散位移  $\chi$  引入 cavity 频移  $\omega_c \rightarrow \omega_c \pm \chi$  (Gao Eq. 33–35):

$$\begin{aligned} H_{\text{int}} &= -\chi \cdot a^\dagger a \cdot \sigma_z / 2 \\ \chi &= 2 \cdot g^2 \cdot \alpha / [\Delta(\Delta + \alpha)] \quad (\text{Gao Eq. 35}) \end{aligned}$$

其中  $g$  是耦合强度,  $\Delta = \omega_q - \omega_c$  是失谐。本项目主要面向**磁通传感** (不是 cavity-based logical qubit), 所以 readout 默认使用 Ramsey-based IQ, cavity 仅在双比特 demo 中出现。

## 2.4 控制脉冲与门

单比特门通过共振微波驱动实现 (Gao §V.B Eq. 49–52):

$$H_d = \Omega(t) \cdot \cos(\omega_d t + \phi) \cdot (a + a^\dagger)$$

在旋转坐标系下 + RWA:

$$H_d \approx (\Omega(t)/2) \cdot (a \cdot e^{i\phi} + a^\dagger \cdot e^{-i\phi})$$

旋转角  $\theta = \int \Omega(t) dt$ 。 $\pi$  脉冲 ( $\theta=\pi$ ) 翻转布居,  $\pi/2$  脉冲 ( $\theta=\pi/2$ ) 制备叠加态。本项目支持 DRAG 脉冲 (Motzoi 2009) 以抑制三能级泄漏。

## 2.5 参数推荐范围 (Gao 2021 §III.B)

| 参数         | 推荐范围        | 本项目默认值   | 物理理由                        |
|------------|-------------|----------|-----------------------------|
| $EJ/h$     | 10–25 GHz   | 15 GHz   | 提供足够强的非线性                   |
| $EC/h$     | 160–400 MHz | 200 MHz  | 保证非谐性可寻址                    |
| $EJ/EC$    | ~50         | ~75      | 抑制电荷噪声                      |
| $f_{01}$   | 4–8 GHz     | ~4.7 GHz | 在 commercial AWG/HEMT 工作带宽内 |
| $\alpha/h$ | 200–300 MHz | 200 MHz  | 单比特门时长 ~10–20 ns 时无泄漏       |

| 参数    | 推荐范围                      | 本项目默认值           | 物理理由              |
|-------|---------------------------|------------------|-------------------|
| $T_1$ | 20–100 $\mu\text{s}$      | 10 $\mu\text{s}$ | 当前 demo 短一些（仿真便利） |
| $T_2$ | $\sim 2 \cdot T_1$ (echo) | 8 $\mu\text{s}$  | 类似                |

**Sanity check:** 任何 PR 引入新 qubit 参数前，须确认  $f_{01} \in [4, 8]$  GHz,  $E_J/E_C \in [40, 80]$ ,  $\alpha/h \in [200, 300]$  MHz；偏离范围必须在 commit message 中说明物理动机。

### 3. 全栈架构

#### 3.1 六层模型（Gao 2021 Fig.1(a)）

|                           |             |                                                |
|---------------------------|-------------|------------------------------------------------|
| workflows/<br>calibration | 顶层科研流程      | ← 串联 experiment + reconstruction +             |
| calibration/              | 标定 workflow | ← 产出 CalibrationTable                          |
| reconstruction/           | 波形重建算法      | ← 消费测量数据，产出重建波形                                |
| experiments/<br>runner    | 实验协议对象      | ← 组合 device + flux + sequence + readout +      |
| simulation/               | QuTiP 调用层   | ← HamiltonianBuilder, MesolveRunner, noise     |
| control/                  | 控制脉冲层       | ← Waveform, FluxSignal, Pulse, gates           |
| hardware/                 | 控制电子学/链路层   | ← ControlLine, DistortionModel, TransferMatrix |
| devices/<br>ChipTopology  | 物理器件层       | ← QubitSpec, TransmonQubit, Resonator,         |

**依赖规则：**上层可依赖下层，禁止反向依赖。例如 `reconstruction` 不可 import `workflows`。这条规则在 CI 中通过 `import-graph` 检查强制（见 §9.4）。

#### 3.2 与 Gao 2021 六层栈的对应

| Gao 2021 层                  | 论文涵盖                    | 本项目 sqc/ 对应                                           | 论文章节    |
|-----------------------------|-------------------------|-------------------------------------------------------|---------|
| Quantum algorithms          | 算法/编译层                  | 不实现（留接口）                                              | §VI     |
| Control software            | Pulse 标定 + 实验序列         | experiments/, calibration/, workflows/                | §V (整章) |
| Control electronics         | AWG, ADC, FPGA          | hardware/electronics.py (stub)                        | §IV.B   |
| Microwave signal processing | IQ mixer, LO, HEMT, 滤波器 | hardware/distortion.py, hardware/readout.py           | §IV.B   |
| Cryogenics + interconnects  | DR, 控制线, 衰减器            | hardware/control_line.py, hardware/transfer_matrix.py | §IV.A   |

| Gao 2021 层 | 论文涵盖                           | 本项目 sqc/ 对应                                            | 论文章节       |
|------------|--------------------------------|--------------------------------------------------------|------------|
| Device     | Transmon, resonator, JJ, SQUID | devices/transmon.py, resonator.py, coupler.py, chip.py | §II + §III |

本项目**重点落在** Device → Control-line transfer function → Microwave signal processing → Calibration → Waveform reconstruction → Predistortion 这条链路。Gao 2021 Fig.1(b) 描述的 "Engineering cycle" 中，本项目仅对应**软件仿真侧** (Hamiltonian design → Simulation → Characterization → 反馈)，**不涉及芯片设计、加工、制冷。**

3.3 目录树

|                             |                                           |
|-----------------------------|-------------------------------------------|
| sqc/                        | ← 主包 (Superconducting Quantum Control)    |
| ├─ __init__.py              |                                           |
| ├─ config.py                | ← 全局配置 (v2.0 新增)                          |
| ├─ devices/                 | ← 物理器件层                                   |
| │   └─ base.py              | ← Device ABC                              |
| │   └─ transmon.py          | ← TransmonQubit, QubitSpec                |
| │   └─ resonator.py         | ← Resonator (= 旧 Cavity)                  |
| │   └─ coupler.py           | ← TunableCoupler ABC                      |
| │   └─ chip.py              | ← ChipTopology, CoupledSystem             |
| ├─ hardware/                | ← 控制电子学/链路层                               |
| │   └─ control_line.py      | ← ControlLine                             |
| │   └─ distortion.py        | ← DistortionModel ABC + 5 子类              |
| │   └─ transfer_matrix.py   | ← TransferMatrix (多 qubit 串扰)             |
| │   └─ electronics.py       | ← AWG/LO 抽象 (P5+)                         |
| │   └─ readout.py           | ← ReadoutModel (Ideal + IQ)               |
| ├─ control/                 | ← 控制脉冲层                                   |
| │   └─ waveform.py          | ← Waveform, CompositeWaveform             |
| │   └─ flux_signal.py       | ← FluxSignal (磁通语义)                       |
| │   └─ pulse.py             | ← Pulse, CompositePulse                   |
| │   └─ sequence.py          | ← PulseSequence + 工厂                      |
| │   └─ gates.py             | ← 单/双比特门 (DRAG/iSWAP/CZ)                  |
| ├─ simulation/              | ← QuTiP 调用层                               |
| │   └─ hamiltonian.py       | ← HamiltonianBuilder (纯函数)                |
| │   └─ runner.py            | ← MesolveRunner, SlidingMeasurementRunner |
| │   └─ noise.py             | ← 1/f 噪声生成                                |
| │   └─ result.py            | ← ExperimentResult, MeasurementTrace      |
| ├─ experiments/             | ← 实验协议对象                                  |
| │   └─ base.py              | ← Experiment ABC                          |
| │   └─ rabi.py              | ← RabiExperiment                          |
| │   └─ ramsey.py            | ← RamseyExperiment                        |
| │   └─ echo.py              | ← DiffEchoExperiment                      |
| │   └─ transient.py         | ← TransientSensingExperiment              |
| │   └─ cryoscope.py         | ← CryoscopeExperiment                     |
| ├─ calibration/             | ← 标定 workflow                             |
| │   └─ base.py              | ← Calibration ABC + CalibrationTable      |
| │   └─ qubit_frequency.py   | ← QubitFrequencyCalibration               |
| │   └─ flux_response.py     | ← FluxResponseCalibration                 |
| │   └─ transfer_function.py | ← TransferFunctionCalibration             |

|                                |                                               |
|--------------------------------|-----------------------------------------------|
| └─ predistortion.py            | ← PredistortionDesigner                       |
| └─ reconstruction/             | ← 波形重建算法                                      |
| └─ base.py                     | ← Reconstruction ABC                          |
| └─ basis.py                    | ← 基函数生成、分解、正则化矩阵                              |
| └─ kernel.py                   | ← KernelEstimator                             |
| └─ wiener.py                   | ← Wiener / RamseyIQ / DiffEcho / RamseyUnwrap |
| └─ hammerstein.py              | ← HammersteinWienerReconstruction             |
| └─ numerical_inverse.py        | ← LMReconstruction                            |
| └─ cryoscope.py                | ← CryoscopeReconstruction                     |
| └─ workflows/                  | ← 顶层科研流程                                      |
| └─ base.py                     | ← Workflow ABC                                |
| └─ predistortion_validation.py |                                               |
| └─ z_crosstalk.py              |                                               |
| src/                           | ← 旧目录，永久冻结（R1 硬约束）                            |
| src_mirror/                    | ← facade 层，重导出自 sqc/                          |
| tests/                         | ← 测试套件                                        |
| docs/                          | ← 本文档所在                                       |
| idea/                          | ← 设计文档（refactor plan、phase handbooks）         |

3.4 模块职责一句话

| 层        | 模块                 | 职责                                                               |
|----------|--------------------|------------------------------------------------------------------|
| devices  | transmon.py        | Transmon 物理参数与算符， <b>不存任何实验状态</b>                                |
|          | resonator.py       | 多模腔模型，张量积算符                                                      |
|          | coupler.py         | 可调耦合器（等效或显式）                                                     |
|          | chip.py            | 多 qubit 芯片拓扑                                                     |
| hardware | control_line.py    | 一条物理控制线（xy/z/readout）的元数据与传递函数                                   |
|          | distortion.py      | AWG→片上的失真模型                                                      |
|          | transfer_matrix.py | 多 qubit 串扰矩阵 $H_{ji}(\omega)$                                    |
|          | readout.py         | 投影/IQ/误判矩阵读出模型                                                   |
| control  | waveform.py        | 通用时域波形（无物理语义）                                                    |
|          | flux_signal.py     | 磁通语义的信号（继承 Waveform）                                             |
|          | pulse.py           | 微波脉冲 + 哈密顿量 list 格式                                              |
|          | sequence.py        | Ramsey/Echo/CPMG/Cryoscope 等脉冲序列工厂                               |
|          | gates.py           | DRAG/iSWAP/CZ 门                                                  |
|          | hamiltonian.py     | 由 device + flux_signal + pulse 构造 QuTiP $H_{list}$ ， <b>无副作用</b> |

| 层              | 模块                                                               | 职责                                                 |
|----------------|------------------------------------------------------------------|----------------------------------------------------|
|                | runner.py                                                        | mesolve/sliding measurement 的执行器                   |
|                | noise.py                                                         | 1/f 噪声生成 + Lindblad 算符                             |
|                | result.py                                                        | 统一结果数据结构                                           |
| experiments    | rabi/ramsey/echo/transient/cryoscope.py                          | 一个实验 = device + flux + sequence + readout + runner |
| calibration    | qubit_frequency/flux_response/transfer_function/predistortion.py | 标定 workflow，产出 CalibrationTable 或 filter 系数        |
| reconstruction | kernel/wiener/hammerstein/numerical_inverse/cryoscope.py         | 仅消费测量数据 + kernel + calibration table，产出重建波形        |
| workflows      | predistortion_validation/z_crosstalk.py                          | 串联多个 experiment + reconstruction + calibration     |

3.5 依赖矩阵（高层只能依赖低层）

|                                   |                                                                                    |                                                        |
|-----------------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------|
| workflows                         | → experiments, calibration, reconstruction, simulation, control, hardware, devices |                                                        |
| calibration                       | → experiments, reconstruction, simulation, hardware, devices                       | （不依赖 workflows）                                        |
| reconstruction                    | → simulation, control, devices                                                     | （不依赖 calibration/experiments/workflows）                |
| calibration/experiments/workflows | → simulation, control, hardware, devices                                           | （不依赖 calibration/reconstruction/workflows）             |
| simulation                        | → control, hardware, devices                                                       | （不依赖 experiments/calibration/reconstruction/workflows） |
| control                           | → devices                                                                          | （允许引用 hardware.distortion 仅作类型注解）                      |
| hardware                          | → devices                                                                          |                                                        |
| devices                           | → （基础库）                                                                            |                                                        |
| config                            | → （基础库）                                                                            | （任何层都可以 import config）                                 |

4. 模块详解

每个子节按以下结构组织：

**物理对应：**该层在 cQED 实验中扮演什么角色，对应 Gao 2021 哪些章节 **关键类：**每个类的用途、构造参数、关键方法 **设计要点：**为什么这样组织，哪些不变量必须保持 **代码示例：**典型用法 **扩展点：**如何添加新功能

4.1 devices/ — 物理器件层

**物理对应：**Gao 2021 §II（量子比特物理）和 §III（芯片设计）。本层描述**孤立器件**——qubit 自身的能级结构、cavity 的模式、芯片的拓扑。**不涉及**外部驱动或测量。

#### 4.1.1 QubitSpec (sqc/devices/transmon.py)

不可变 (frozen dataclass) 的 Transmon 参数描述对象，**不存储任何实验状态**。这是本项目最基础的物理对象。

```
from sqc.devices.transmon import QubitSpec
import numpy as np

spec = QubitSpec(
    name="Q0",
    EC=2 * np.pi * 0.2, # rad·GHz
    EJ=2 * np.pi * 15, # rad·GHz at zero flux
    T1=10000.0, # ns
    T2=8000.0, # ns
    flux_bias=0.0, #  $\Phi_0$ 
    n_levels=3, # Fock 截断
)

f01 = spec.frequency() #  $f_{01}$  at current flux
f01_at_flux = spec.frequency(0.1) #  $f_{01}$  at  $\Phi=0.1 \Phi_0$ 
kappa = spec.sensitivity() #  $dw/d\Phi$  at current flux
new_spec = spec.with_flux(0.05) # return NEW QubitSpec, does NOT mutate
```

关键方法：

| 方法                                         | 物理含义                                                     | Gao 2021 公式     |
|--------------------------------------------|----------------------------------------------------------|-----------------|
| <code>frequency(flux=None)</code>          | $f_{01}(\Phi) = \sqrt{(8 \cdot EJ(\Phi) \cdot EC) - EC}$ | Eq. 18          |
| <code>anharmonicity()</code>               | $\alpha = -EC$                                           | Eq. 18          |
| <code>sensitivity(flux, delta=1e-6)</code> | $\kappa = df_{01}/d\Phi$ (中心差分)                          | —               |
| <code>with_flux(new_flux)</code>           | 返回新对象 (不可变模式)                                            | —               |
| <code>EJ_at(flux=None)</code>              | $EJ(\Phi) = EJ_0 \cdot \cos(\pi\Phi)$                    | $\cos(\pi\Phi)$ |
| <code>optimal_work_point()</code>          | 最大灵敏度点 (静态方法)                                            | —               |

设计要点：

- `frozen=True` 强制不可变性。任何“改变 flux 后的 qubit”都要通过 `with_flux()` 返回新对象。
- 这消除了重构前 D1 债务 (`qubit_in_mag` 副作用) 的根因。

#### 4.1.2 TransmonQubit (sqc/devices/transmon.py)

向后兼容的完整 qubit 类，内部包装 `QubitSpec`。保留所有 `src/qubit.py` 的方法签名。

```
from sqc.devices.transmon import TransmonQubit

q = TransmonQubit(EC=2*np.pi*0.2, EJ=2*np.pi*15, T1=10000, T2=8000)
q.qubit_in_mag(flux_signal) # 计算 H_list (副作用: 设置 q.H_list)
H = q.get_hamiltonian() # 静态哈密顿量
H_rwa = q.get_hamiltonian_rwa(omega_d) # 旋转波近似哈密顿量
c_ops = q.get_collapse_operators() # Lindblad 坍缩算符
```



为什么保留 **mutable** 接口：src/protocal.py 等 legacy 代码 heavily 依赖 qubit.H\_list, qubit.mag\_signal 等 instance attributes。TransmonQubit 通过 wrapping QubitSpec 同时支持"老式"和"新式"两种用法，新代码推荐用 QubitSpec + HamiltonianBuilder。

坍塌算符模型 (Gao §V.B.2 简化)：

```
c_ops = [√γ1·a, √γφ·n]
γ1 = 1/T1, γφ = 1/T2 - γ1/2
```

#### 4.1.3 Resonator (sqc/devices/resonator.py)

多模 LC 谐振腔模型。替代旧 Cavity 类 (src/qubit.py)。

```
from sqc.devices.resonator import Resonator

cav = Resonator(frequencies=[7.0, 7.5], n_levels=[3, 3]) # 双模 cavity
H_cav = cav.get_hamiltonian()
```

#### 4.1.4 ChipTopology / CoupledSystem (sqc/devices/chip.py)

多 qubit 芯片拓扑容器。管理 qubit 列表、谐振腔列表、耦合映射、控制线分配。

```
from sqc.devices.chip import ChipTopology

chip = ChipTopology(
    qubits=[q0, q1],
    resonators=[resonator],
    couplings={("Q0", "R0"): 0.05, ("Q1", "R0"): 0.05},
)
op_lifted = chip.lift_qubit_op(sigma_z, "Q0") # 嵌入完整 Hilbert 空间
H_chip = chip.hamiltonian_static()
```

CoupledSystem 是 ChipTopology 的子类，专门处理双 qubit + 单 cavity 的情况 (旧 Coupled\_System 兼容包装)。

#### 4.1.5 扩展点

- 添加新 qubit 类型 (如 Fluxonium)：新建 sqc/devices/fluxonium.py，继承 Device ABC，实现 hilbert\_dim(), hamiltonian\_static(), collapse\_operators()。
- 添加新耦合器 (如可调 SQUID coupler)：新建 sqc/devices/coupler.py 中的子类，实现耦合强度的时变接口。

## 4.2 hardware/ — 控制电子学层

**物理对应**：Gao 2021 §IV (设置测量)。这一层处理 AWG 输出到芯片之间的所有 **imperfection**：阻抗失配、传输线滤波、控制线串扰、IQ 解调读出。**关键洞察**：在仿真中，这些 imperfection 通过 DistortionModel 和 TransferMatrix 显式建模，正如真实实验中通过 Cryoscope 标定和补偿。

#### 4.2.1 ControlLine (sqc/hardware/control\_line.py)

一条物理控制线（xy/z/readout）的完整描述。

```
from sqc.hardware.control_line import Controlline
from sqc.hardware.distortion import SingleExponentialDistortion

line = Controlline(
    name="Z0", kind="z", source="AWG0:CH1", target="Q0",
    transfer_function=SingleExponentialDistortion(amplitude=0.01, tau=50.0),
)
on_chip = line.apply(awg_waveform)          # AWG → 片上（前向失真）
awg = line.predistort(target, designer)      # 片上目标 → AWG 波形（逆滤波）
```

字段：

- **name**: 标识符，如 "Z0", "XY1"
- **kind**: "xy" | "z" | "readout"
- **source**: AWG 通道，如 "AWG0:CH1"
- **target**: 芯片节点，如 "Q0", "C01"
- **transfer\_function**: 一个 `DistortionModel` 实例

4.2.2 DistortionModel (sqc/hardware/distortion.py)

ABC + 5 个子类，建模 AWG→片上的失真：

| 模型                          | 描述         | 参数                   | 典型物理意义        |
|-----------------------------|------------|----------------------|---------------|
| SingleExponentialDistortion | 单指数尾巴（最常用） | amplitude, tau       | 慢电荷再分布尾巴      |
| MultiExponentialDistortion  | 多时间常数尾巴    | amplitudes[], taus[] | 多 RC 时间常数     |
| FIRDistortion               | 有限冲激响应滤波器  | taps[]               | 通用线性预失真后端     |
| IIRDistortion               | 无限冲激响应滤波器  | b[], a[]             | bias-tee 高通响应 |
| CustomTransferDistortion    | 自定义频域 H(ω) | omega_grid, H_grid   | 测得的频响曲线       |

所有模型实现的 ABC 方法：

| 方法                        | 签名                                | 物理含义                                 |
|---------------------------|-----------------------------------|--------------------------------------|
| apply(waveform, dt)       | (np.ndarray, float) → np.ndarray  | $y[n] = \sum h[k] \cdot x[n-k]$ 离散卷积 |
| step_response(t)          | np.ndarray → np.ndarray           | s(t)：阶跃输入下的输出                        |
| impulse_response(t)       | np.ndarray → np.ndarray           | h(t)：冲激输入下的输出（核函数）                   |
| frequency_response(omega) | np.ndarray → np.ndarray (complex) | H(ω)：复频域响应                           |
| apply_to_waveform(wf)     | Waveform → Waveform               | 便捷接口，自动处理 dt                         |

关键示例：单指数模型

$$h(t) = (1 - \text{amp}) \cdot \delta(t) + (\text{amp}/\tau) \cdot \exp(-t/\tau) \cdot \Theta(t)$$
$$H(\omega) = (1 - \text{amp}) + \text{amp} / (1 + j\omega\tau)$$

### 4.2.3 TransferMatrix (sqc/hardware/transfer\_matrix.py)

多 qubit Z 线串扰矩阵:  $\Phi_i(\omega) = \sum_j H_{ji}(\omega) \cdot V_j(\omega)$  (Gao §V.E)。

```
from sqc.hardware.transfer_matrix import TransferMatrix

tm = TransferMatrix.from_dc_matrix(
    dc_matrix=np.array([[1.0, 0.05],
                        [0.03, 1.0]]),
    source_names=["QA", "QB"],
    target_names=["QA", "QB"],
)
fluxes = tm.apply({"QA": pulse_on_A, "QB": pulse_on_B})
phi_B = fluxes["QB"] # 含来自 QA 的串扰

# 频率相关版本 (不只是 DC 矩阵)
tm_freq = TransferMatrix(
    elements={
        ("QA", "QA"): H_AA_omega, # 复数数组
        ("QB", "QA"): H_BA_omega, # off-diagonal 串扰
        ...
    },
    frequency_axis=omega_grid,
)
```

关键方法:

- `apply(source_voltages)`: FFT 输入 → 频域乘 H → IFFT → 片上 fluxes
- `H_ji(target, source)`: 访问单元素
- `diagonal() / off_diagonal()`: 分离对角与非对角
- `from_dc_matrix()`: 从 DC 串扰矩阵构造频率平坦的 TransferMatrix

### 4.2.4 ReadoutModel (sqc/hardware/readout.py)

读出层抽象。两个具体实现:

- **IdealProjectiveReadout**: 投影测量到  $|1\rangle$ , 返回 `{"p_e": float}`
- **IQReadoutModel**: IQ 解调读出。运行两个 Ramsey 序列 (相位 0 和  $\pi/2$ ), 返回 `{"p_e_I": ..., "p_e_Q": ...}`

```
from sqc.hardware.readout import IdealProjectiveReadout, IQReadoutModel

ro1 = IdealProjectiveReadout()
result = ro1.measure(final_state, qubit=q)
print(result["p_e"])

ro2 = IQReadoutModel(tau=20.0)
qubit.qubit_in_mag(some_flux_signal)
result = ro2.measure(qubit)
varphi = np.arctan2(result["p_e_Q"] - 0.5, result["p_e_I"] - 0.5)
```

### 4.2.5 扩展点

- 添加新失真模型：继承 `DistortionModel`，实现 4 个抽象方法。详见 §7.3。
- 添加单次读出 (**single-shot**)：新建 `SingleShotReadout(ReadoutModel)`，实现 IQ 平面散射与误判矩阵。
- 添加新串扰建模：例如时变 `TransferMatrix`，可继承现有 `TransferMatrix` 添加 `apply_at_time(t)` 方法。

## 4.3 control/ — 控制脉冲层

**物理对应：**Gao 2021 §IV.B (IQ 调制驱动) + §V.B (单比特门设计)。这一层只关心“AWG 应该输出什么样的波形”——既不知道 qubit 物理（那是 devices 层），也不知道失真（那是 hardware 层）。

### 4.3.1 Waveform (`sqc/control/waveform.py`)

通用时域波形（无物理语义）。是所有信号类的基础。

```
from sqc.control.waveform import Waveform

wf = Waveform(
    t_list=np.linspace(0, 100, 1000),
    samples=np.sin(2*np.pi*0.01*t),
)
wf.value_at(25.0)          # 采样保持插值
wf.truncate(20, 80)        # 返回新对象，边缘置零
wf.plot()                  # matplotlib 可视化
wf.duration                 # property: 最后 - 第一时刻
wf.n_points                 # property: 时间点数
```

`CompositeWaveform` 是 `Waveform` 的子类，用于拼接多个波形序列。

### 4.3.2 FluxSignal (`sqc/control/flux_signal.py`)

磁通信号（继承 `Waveform`，samples 单位为  $\Phi_0$ ）。兼容旧 `Signal(type=N, ...)` 构造方式。

```
from sqc.control.flux_signal import FluxSignal

# type-based 构造（兼容旧 API）
phi = FluxSignal(type=2, t_list=t, amplitude=0.001, frequency=0.01) # 正弦
phi = FluxSignal(type=3, t_list=t, amplitude=0.01, center=50, width=3) # 高斯
phi = FluxSignal(type=8, t_list=t, signal=custom_array) # 自定义
```

**类型映射**（旧 type → 新 kind）：

| type | kind               | 描述     | 关键参数                             |
|------|--------------------|--------|----------------------------------|
| 0    | "zero"             | 零信号    | (无)                              |
| 1    | "constant"         | 常数     | amplitude                        |
| 2    | "sinusoidal"       | 正弦波    | amplitude, frequency, rise, fall |
| 3    | "gaussian"         | 高斯脉冲   | amplitude, center, width         |
| 4    | "asymmetric_pulse" | 非对称双指数 | amplitude, rise, fall            |

| type | kind              | 描述    | 关键参数                                |
|------|-------------------|-------|-------------------------------------|
| 5    | "double_peak"     | 双峰    | amplitude, centers, widths          |
| 6    | "basis_expansion" | 基函数展开 | basis_type, n_basis, b              |
| 7    | "wavepacket"      | 波包    | amplitude, frequency, width, center |
| 8    | "custom"          | 自定义数组 | signal (np.ndarray)                 |

#### 4.3.3 Pulse / CompositePulse (sqc/control/pulse.py)

微波控制脉冲。支持 lab frame / rotating frame + RWA。

```
from sqc.control.pulse import Pulse, CompositePulse
from sqc.control.flux_signal import FluxSignal as Signal

Omega = Signal(type=3, t_list=t, amplitude=1.0, center=t_mid, width=sigma)
pulse = Pulse(
    Omega=Omega,
    frame=1,                      # 1 = rotating, 0 = lab
    omega_d=qubit.frequency,
    phase=0.0,
    is_rwa=True,
    qubit=qubit,
)
H_pulse = pulse.hamiltonian      # QuTiP list format
```

**CompositePulse** 拼接多个 **Pulse**，处理时间偏移与哈密顿量合并。

#### 4.3.4 序列工厂 (sqc/control/sequence.py)

```
from sqc.control.sequence import (
    create_pulse,           # 单脉冲
    create_ramsey_pulse,    #  $\pi/2 - \tau - \pi/2$ 
    create_echo_pulse,      #  $\pi/2 - \tau - \pi - \tau - \pi/2$ 
    create_diff_echo_pulse, #  $\pi/2 - [\text{echo}]^k - \pi/2$ 
    create_cpmg_pulse,      # CPMG 序列
    create_cryoscope_pulse, #  $Y/2 - Z(t) - Y/2$ 
)

t_rabi = CONFIG.pulse.t_rabi
ramsey = create_ramsey_pulse(t_rabi, tau=100.0, omega_d=q.frequency, qubit=q)
```

**重要：**所有自由演化空隙（**Omega\_0** 段）的时间轴使用 **CONFIG.awg.dt** 派生（v2.0 全局配置），不再硬编码 **np.linspace(0, tau, 100)**。

#### 4.3.5 门函数 (sqc/control/gates.py)

```
from sqc.control.gates import ideal_iSWAP, simulate_iSWAP, ideal_CZ, simulate_CZ
U_iswap = ideal_iSWAP()
```

```
fidelity, U_sim = simulate_iSWAP(qubit1, qubit2, g, duration)
```

### 4.3.6 扩展点

- 添加新的 **FluxSignal type**: 在 `_generate()` 函数中添加新的 case; 在 `_fill_default_params()` 中添加默认参数。详见 §7.4。
- 添加新的脉冲序列工厂: 在 `sequence.py` 中按照 `create_ramsey_pulse` 的模板新建函数, 返回 `CompositePulse`。
- 添加新的门函数: 在 `gates.py` 中按 `ideal_iSWAP/simulate_iSWAP` 的模板。

## 4.4 simulation/ — 仿真层

**物理对应**: 这一层不对应 Gao 论文的某个特定章节——它是项目自有的**数值核心**, 负责把上层的物理对象翻译成 QuTiP 的输入, 并执行时间演化。**关键设计**: 无副作用、单一职责。

### 4.4.1 HamiltonianBuilder (`sqc/simulation/hamiltonian.py`)

**纯函数**: 从 device + flux + pulse 构造 QuTiP `H_list`, 不修改任何输入对象。

```
from sqc.simulation.hamiltonian import HamiltonianBuilder

H_list, t_global = HamiltonianBuilder.build(
    qubit=spec,                # QubitSpec 或 TransmonQubit
    flux_signal=phi,           # FluxSignal 或 None
    pulse=ramsey_pulse,        # PulseBase 或 None
    frame="rotating",          # "lab" or "rotating"
    omega_d=qubit.frequency,
)
# H_list 直接传给 QobjEvo 或 mesolve
```

**返回的 `H_list` 格式** (QuTiP 标准 list 格式):

```
H_list = [
    H_anharmonic,              # 不随时间变化项
    [n_op, freq_coeffs_array], # 随磁通时变项 (rotating frame:  $\omega(t) - \omega_d$ )
    [op1, coeff1_array],       # 脉冲驱动项
    ...
]
```

### 4.4.2 MesolveRunner / SlidingMeasurementRunner (`sqc/simulation/runner.py`)

```
from sqc.simulation.runner import MesolveRunner, SlidingMeasurementRunner

runner = MesolveRunner()
result = runner.run(H_list, psi0, t_list, c_ops, e_ops)

# 用于瞬态传感 (case 4) 的滑动测量
```

```
sliding = SlidingMeasurementRunner()
result = sliding.run(qubit, flux_signal, control_pulse, scan_list)
```

`MesolveRunner.run()` 是对 `qutip.mesolve` 的轻量封装，返回标准化的 `ExperimentResult`。

`SlidingMeasurementRunner.run()` 实现瞬态传感的关键算法：把控制脉冲沿磁通信号滑动，记录每个位置的 `p_e`。

#### 4.4.3 noise (sqc/simulation/noise.py)

```
from sqc.simulation.noise import generate_1f_noise

noise = generate_1f_noise(t_list, amplitude=1e-4, f_min=0.001, f_max=1.0, seed=42)
```

1/f 噪声生成。可叠加到 `FluxSignal.samples` 上模拟环境噪声。

#### 4.4.4 ExperimentResult (sqc/simulation/result.py)

统一的实验结果容器，支持 pickle 序列化。

```
@dataclass
class ExperimentResult:
    data: dict[str, np.ndarray] # 命名数据: p_e, kernel, delta_p, ...
    axes: dict[str, np.ndarray] # 命名轴: tau, scan, t_samples, ...
    metadata: dict              # qubit_spec, experiment_class, git_sha, ...
    config: dict                 # 调用时的输入参数

result.save("output.pkl")
result = ExperimentResult.load("output.pkl")
```

辅助函数：

- `extract_expectation(qutip_result, idx)`：从 `qutip.Result` 提取 `.expect[idx]`
- `extract_population(qutip_result, level)`：从 `.states` 计算 `|level>` 的布居

#### 4.4.5 扩展点

- 添加新 Runner（如 `MCSolveRunner`、`MasterEquationRunner`）：继承 `RunnerBase`，实现 `run()`。
- 添加新噪声类型（如电报噪声、白噪声）：在 `noise.py` 中添加新函数。

### 4.5 experiments/ — 实验协议层

**物理对应：** Gao 2021 §V.A–V.D (spectroscopy + 门表征流程)。每个 `Experiment` 类对应一个标准 cQED 实验协议。

#### 4.5.1 通用结构

每个 `Experiment` 类 = device + flux\_signal + sequence + readout + runner 的组合，继承 `Experiment ABC`：

```
from dataclasses import dataclass, field

@dataclass
class Experiment(ABC):
    @abstractmethod
    def build_sequence(self) -> PulseSequence: ...

    @abstractmethod
    def run(self) -> ExperimentResult: ...
```

4.5.2 五个具体实验

| Experiment                 | 描述                                      | Gao 章节        | 输出关键字段                          |
|----------------------------|-----------------------------------------|---------------|---------------------------------|
| RabiExperiment             | 扫描脉冲时长，观察 p_e 振荡                        | §V.B.1        | result.expect[0] (qutip.Result) |
| RamseyExperiment           | $\pi/2 - \tau - \pi/2$ ，测相位累积           | §V.B.2 Eq. 54 | data["p_e"], axes["tau"]        |
| DiffEchoExperiment         | $\pi/2 - [\text{echo}]^k - \pi/2$ ，差分回波 | §V.B.2        | data["p_e"], k, t_int           |
| TransientSensingExperiment | 滑动 Ramsey + kernel 提取                   | (项目原创)        | data["kernel"], data["delta_p"] |
| CryoscopeExperiment        | 截断扫描 + IQ 测相位                           | §V.E          | data["varphi"], axes["trunc"]   |

4.5.3 RamseyExperiment 详例

```
from sqc.experiments.ramsey import RamseyExperiment

exp = RamseyExperiment(
    qubit=q,
    flux_signal=phi, # 可选，None 时用默认正弦
    tau_list=np.linspace(0, 250, 500),
)
result = exp.run()
# result.data["p_e"]      → Ramsey 干涉条纹 (shape=(len(tau_list),))
# result.axes["tau"]      → 自由演化时间轴
# result.data["flux_samples"] → 磁通信号快照
```

默认参数 (v2.0 起从 CONFIG.pulse 派生):

- t\_rabi: CONFIG.pulse.t\_rabi (20 点, 0..9.5 ns, dt=0.5 ns)
- tau\_list: CONFIG.pulse.tau\_list (500 点, 0..249.5 ns)
- t\_global: CONFIG.pulse.t\_global (900 点, -50..399.5 ns)

4.5.4 TransientSensingExperiment 详例



```
from sqc.experiments.transient import TransientSensingExperiment

exp = TransientSensingExperiment(qubit=q, flux_signal=phi)
result = exp.run()
# result.data["kernel"]    → 控制核函数 K(t)
# result.data["delta_p"]   → 测量 Δp 数组
# result.axes["scan"]     → 滑动扫描位置
```

后接 `WienerReconstruction.reconstruct()` 可恢复  $\Phi(t)$ 。

4.5.5 扩展点

详见 §7.1。简言之：继承 `Experiment` ABC，实现 `build_sequence()` 和 `run()`，从 `CONFIG.pulse` 获取默认时间轴。

4.6 reconstruction/ — 波形重建层

**物理对应：**Gao 2021 §V.E 简略提及 Cryoscope；本项目的 reconstruction 层是**研究创新**，与论文正交。

4.6.1 算法总览

所有重建类继承 `Reconstruction` ABC，实现 `reconstruct(measurement, kernel, calibration)`：

| 类                               | 算法                          | 物理基础                                               | 输入 → 输出                                        |
|---------------------------------|-----------------------------|----------------------------------------------------|------------------------------------------------|
| WienerReconstruction            | 线性 Wiener 反卷积               | $\Delta p = K * \Phi + \text{noise}$               | $\Delta p + \text{kernel} \rightarrow \Phi(t)$ |
| HammersteinWienerReconstruction | 块结构非线性：Wiener + 色散反演        | 加入 $\omega(\Phi)$ 非线性                              | $\Delta p + \text{kernel} \rightarrow \Phi(t)$ |
| LMReconstruction                | Levenberg-Marquardt 全密度矩阵优化 | 直接 fit $p_{\text{meas}}$                           | $p_{\text{meas}} \rightarrow \Phi(t)$ (基函数参数化) |
| RamseyIQReconstruction          | IQ 解调 → arcsin → $B(\tau)$  | $\varphi(\tau) = \kappa \cdot \int \Phi \, dt$     | $(I, Q) \rightarrow B(\tau)$                   |
| RamseyUnwrapReconstruction      | 相位解缠绕                       | 同上，但单通道                                            | $p_e(\tau) \rightarrow B(\tau)$                |
| DiffEchoReconstruction          | 差分回波直接公式                    | $B = -\varphi / (2\kappa \cdot \tau_{\text{int}})$ | $p_{e\_list} \rightarrow B$                    |
| CryoscopeReconstruction         | 相位微分 + $\varphi(h)$ 反函数     | $d\varphi/dt \times \tau_{\text{cal}} = \Phi$      | $\varphi(\text{trunc}) \rightarrow \Phi(t)$    |

4.6.2 KernelEstimator (sqc/reconstruction/kernel.py)

统一的控制核函数估计器（消除了旧代码三处重复实现）。

```
from sqc.reconstruction.kernel import KernelEstimator

estimator = KernelEstimator(stim_amplitude=0.0215, stim_width=3.0)
t_samples, kernel = estimator.estimate(pulse, qubit)
```

**算法：**在每个时间点  $t_i$  注入窄高斯刺激，测量  $p_e$  变化。 $\text{kernel}[i] = (p_{e\_stimulated} - p_{e\_baseline}) / \text{stim\_area}$ 。

#### 4.6.3 WienerReconstruction 详例

```
from sqc.reconstruction.wiener import WienerReconstruction

recon = WienerReconstruction(lambda_reg=1.0)
phi_rec = recon.reconstruct(
    measurement=transient_result,    # ExperimentResult with data["delta_p"]
    kernel=kernel_array,
    dt=0.5,
)
# phi_rec.samples → 重建的  $\Phi(t)$ 
```

**公式：** $H_{\text{inv}}(\omega) = K^*(\omega) / (|K(\omega)|^2 + \lambda^2)$ ，对  $\Delta p(\omega)$  应用  $\rightarrow \Phi(\omega) \rightarrow \text{IFFT}$  回时域。

#### 4.6.4 LMReconstruction 详例

```
from sqc.reconstruction.numerical_inverse import LMReconstruction

recon = LMReconstruction(
    qubit=q, control_pulse=cp,
    basis_type="fourier", n_basis=100,
    lambda_reg=100.0, max_iter=10, tol=1e-6,
    use_adjoint=True,          # True: 伴随法 Jacobian (快); False: 有限差分
)
phi_rec, history = recon.reconstruct(measurement)
```

**算法：**把  $\Phi(t)$  参数化为基函数  $b \cdot \phi_k(t)$ ，定义损失  $L(b) = \|p_{\text{sim}}(b) - p_{\text{meas}}\|^2 + \lambda^2 \cdot \|R \cdot b\|^2$ ，用 LM 优化  $b$ 。

#### 4.6.5 CryoscopeReconstruction 详例

```
from sqc.reconstruction.cryoscope import CryoscopeReconstruction

# 先做  $\phi(h)$  标定
cal_table = FluxResponseCalibration(qubit=q, method="cryoscope").calibrate()

recon = CryoscopeReconstruction(calibration=cal_table, tau=100.0, method="calib_inverse")
flux_rec = recon.reconstruct(cryoscope_result)
```

**注：**`method="calib_inverse" / "SG_diff" / "diff"` 三种反演策略，依赖 Track B 1.1 完成（当前为 stub）。

#### 4.6.6 basis 模块 (sqc/reconstruction/basis.py)

```
from sqc.reconstruction.basis import (
    generate_basis_functions,    # 生成基函数列表 (Fourier/B-spline/Legendre)
    basis_function_decomposition, # 信号 → 基函数系数
```

```
regularization_matrix,          # R 矩阵（光滑度正则）
)
```

被 `LMReconstruction` 和 `FluxSignal(type=6)` 使用。

4.6.7 扩展点

详见 §7.2。简言之：继承 `Reconstruction ABC`，实现 `reconstruct(measurement, kernel, calibration, **kwargs)`。

4.7 calibration/ — 标定层

**物理对应：**Gao 2021 §V.A–V.C 整章。标定的目标是把"未知物理参数"变成"已知数值"，是任何科研实验的前置步骤。

4.7.1 通用结构

```
class Calibration(ABC):
    @abstractmethod
    def __init__(self, qubit: TransmonQubit, **kwargs): ...

    @abstractmethod
    def calibrate(self) -> CalibrationTable: ...
```

所有标定类返回 `CalibrationTable`：

```
@dataclass(frozen=True)
class CalibrationTable:
    qubit_name: str
    kind: Literal["f01", "f_phi", "kappa", "phi_h", "transfer_function"]
    inputs: np.ndarray          # 自变量
    outputs: np.ndarray         # 因变量
    fit_params: dict
    metadata: dict

    def evaluate(self, x): ...   # 插值
    def inverse(self, y): ...    # 反函数（如可逆）
```

4.7.2 具体标定类

| Calibration                                              | 物理目标            | Gao 章节 | 方法                                 |
|----------------------------------------------------------|-----------------|--------|------------------------------------|
| <code>QubitFrequencyCalibration</code>                   | 标定 $f_{01}$     | §V.A   | Ramsey 自由进动 + FFT                  |
| <code>FluxResponseCalibration(method="ramsey")</code>    | $f(\Phi)$ 曲线    | §V.A   | 扫描 DC flux + Ramsey                |
| <code>FluxResponseCalibration(method="cryoscope")</code> | $\varphi(h)$ 标定 | §V.E   | 扫描方波高度 + IQ Ramsey（依赖 Track B 1.1） |

| Calibration                                 | 物理目标                     | Gao 章节   | 方法                                         |
|---------------------------------------------|--------------------------|----------|--------------------------------------------|
| FluxResponseCalibration(method="transient") | $\Delta\omega(\Phi)$ 多项式 | (项目原创)   | 已知瞬态信号扫描（依赖 Track B 1.2）                   |
| TransientFrequencyCalibration               | 多项式系数                    | (项目原创)   | 同上（stub，依赖 Track B 1.2）                    |
| TransferFunctionCalibration                 | $H(\omega)$ 拟合           | $\S V.E$ | Cryoscope 阶跃响应 + 多指数拟合                     |
| PredistortionDesigner                       | 设计逆滤波器                   | $\S V.E$ | $H_{\text{inv}}(\omega) = H^*(\omega) / ($ |

4.7.3 QubitFrequencyCalibration 详例

```
from sqc.calibration.qubit_frequency import QubitFrequencyCalibration

cal = QubitFrequencyCalibration(qubit=q)
table = cal.calibrate()
print(table.outputs[0])    # 拟合得到的 f_01 (rad·GHz)
print(table.fit_params["contrast"])
```

算法：

- 1. 在零磁通下运行 Ramsey（扫描 tau）
- 2. FFT 找  $p_e(\text{tau})$  的主频，得到 detuning
- 3.  $f_{01} = \omega_d + \text{detuning}$

4.7.4 PredistortionDesigner 详例

```
from sqc.calibration.predistortion import PredistortionDesigner
from sqc.hardware.distortion import SingleExponentialDistortion

# 已知失真模型
dist = SingleExponentialDistortion(amplitude=0.05, tau=20.0)

# 设计补偿
designer = PredistortionDesigner(method="fir_inverse", n_taps=64, regularization=1e-4)
inverse_model = designer.design(dist, dt=0.5)

# 应用预失真
predistorted = designer.predistort(target_waveform, dist)

# 验证：失真(预失真(target)) ≈ target
corrected = dist.apply_to_waveform(predistorted)
```

算法：

- `method="frequency_inverse"`:  $H_{\text{inv}}(\omega) = H^*(\omega) / (|H|^2 + \lambda^2)$
- `method="fir_inverse"`: IFFT  $H_{\text{inv}}$  得到时域 taps，截断至 `n_taps` 点

- `method="iir_inverse"`: 拟合  $H_{\text{inv}}$  为 IIR (极点稳定性检查)

#### 4.7.5 扩展点

- 添加新标定类型: 继承 `Calibration ABC`, 实现 `calibrate()`, 返回 `CalibrationTable`。
- 改用真实实验数据替代仿真: 在 `Calibration.calibrate()` 中改为读取实际测量数据 (pickle/CSV), 不再调用 `mesolve`。

### 4.8 workflows/ — 顶层科研流程

**物理对应:** Gao 2021 §V Fig. 9 (完整 cQED 表征工作流)。这一层把 experiments + calibration + reconstruction 串成端到端科研流程。

#### 4.8.1 PredistortionValidationWorkflow

```
from sqc.workflows.predistortion_validation import PredistortionValidationWorkflow

wf = PredistortionValidationWorkflow(
    qubit=q,
    target_waveform=target,
    true_distortion=dist,          # ground truth (仿真已知)
)
results = wf.run()
print(results["metrics"]["improvement_factor"]) # 期望 > 10x
```

流程:

1. 用 `true_distortion` 注入失真到 `Controlline`
2. 用 `CryoscopeExperiment` 测得阶跃响应
3. 用 `TransferFunctionCalibration` 拟合  $H(\omega)$
4. 用 `PredistortionDesigner` 设计逆滤波器
5. 应用预失真 → 重新测量 → 计算 RMSE 改善

#### 4.8.2 ZCrosstalkWorkflow

```
from sqc.workflows.z_crosstalk import ZCrosstalkWorkflow

wf = ZCrosstalkWorkflow(
    chip=chip,
    flux_pulse_on_A=pulse,
    true_transfer_matrix=tm,      # ground truth
    qubit_A_name="QA",
    qubit_B_name="QB",
)
results = wf.run()
# results["H_BA_estimated"] → 估计的 QA→QB 串扰
# results["compensation_factor"] → 补偿后寄生相位下降倍数
```

流程:

1. 在 QA 的 Z 线上施加 flux 脉冲
2. 在 QB 上运行 TransientSensingExperiment 测寄生磁通
3. Wiener 反卷积重建  $\Phi_B(t)$
4. 频域解卷积:  $H_{BA}(\omega) = \Phi_B(\omega) \cdot V_A^*(\omega) / (|V_A|^2 + \lambda^2)$
5. 设计补偿脉冲, 重新测量验证

### 4.8.3 扩展点

- 添加新顶层 **workflow** (如完整 RB workflow、双比特门优化 workflow): 继承 **Workflow ABC**, 实现 `run()` → `dict`。
- 修改现有 **workflow** 的某一步: 直接覆写对应的子调用, 例如把 **TransferFunctionCalibration** 换成自定义算法。

## 5. 数据流

### 5.1 完整数据流图 (端到端)



### 5.2 标定辅助流

```

Calibration.calibrate(qubit)
├─ QubitFrequencyCalibration → CalibrationTable(f01)
└─ FluxResponseCalibration  → CalibrationTable(φ vs h)

```

```

└─ TransferFunctionCalibration → CalibrationTable(H( $\omega$ ))
└─ PredistortionDesigner → DistortionModel(逆滤波器)

```

### 5.3 完整端到端 Z-crosstalk 流程

```

ChipTopology + TransferMatrix (ground truth)
├─ apply flux_pulse on QA's Z-line
│   └─ on-chip fluxes { $\Phi_A$ ,  $\Phi_B$ }
└─
    ▼
TransientSensingExperiment(qubit=QB, flux= $\Phi_B$ )
    │
    ▼
WienerReconstruction
    │ →  $\Phi_{B\_estimated}$ 
    │
    ▼
freq-domain deconvolution ( $\Phi_{B\_est} \times V_{A^*}$ ) / ( $|V_A|^2 + \lambda^2$ )
    │ →  $H_{BA}(\omega)$  estimated
    │
    ▼
PredistortionDesigner.design( $H_{BA\_est}$ )
    │ → compensation pulse
    │
    ▼
Apply compensation → verify reduced  $\Phi_B$ 
    └─ → compensation_factor metric

```

### 5.4 跨层调用顺序

```

1. 配置: CONFIG = Config()          (config layer)
2. 构造 qubit: QubitSpec(...)       (devices)
3. 构造信号: FluxSignal(type=...)   (control)
4. 构造序列: create_ramsey_pulse(...) (control)
5. 构造 H_list: HamiltonianBuilder.build(...) (simulation)
6. 演化: MesolveRunner.run(...)     (simulation)
7. 封装: ExperimentResult(...)      (simulation)
8. 反演: WienerReconstruction.reconstruct(...) (reconstruction)
9. 标定 (可选): Calibration.calibrate() (calibration)
10. 流程 (可选): Workflow.run()     (workflows)

```

## 6. 快速开始

### 6.1 环境配置

```

pip install -r requirements.txt
python -c "import qutip, numpy, scipy, matplotlib; print('ok')"

```

或使用 conda 环境：

```
conda activate qutip-env
python -c "import qutip, numpy, scipy, matplotlib; print('ok')"
```

## 6.2 基础用法

```
import numpy as np
from sqc.devices.transmon import TransmonQubit, QubitSpec
from sqc.control.flux_signal import FluxSignal
from sqc.experiments.ramsey import RamseyExperiment

# 1. 创建 qubit
q = TransmonQubit(EC=2*np.pi*0.2, EJ=2*np.pi*15, T1=10000, T2=8000)

# 2. 创建磁通信号
phi = FluxSignal(type=2, t_list=np.linspace(0, 250, 500),
                 amplitude=0.001, frequency=0.01)

# 3. 运行 Ramsey 实验
exp = RamseyExperiment(qubit=q, flux_signal=phi)
result = exp.run()
print(result.data["p_e"]) # Ramsey 干涉条纹
```

## 6.3 使用新架构（推荐）

```
from sqc.devices.transmon import QubitSpec
from sqc.simulation.hamiltonian import HamiltonianBuilder

spec = QubitSpec("Q0", EC=2*np.pi*0.2, EJ=2*np.pi*15, T1=1e4, T2=8e3)
H_list, t_list = HamiltonianBuilder.build(spec, flux_signal=phi)
# spec 未被修改 ✓
```

## 6.4 使用全局配置（v2.0 推荐）

```
from sqc.config import CONFIG
from sqc.devices.transmon import TransmonQubit
from sqc.experiments.ramsey import RamseyExperiment

# 所有参数从 CONFIG 派生
q = TransmonQubit(**CONFIG.transmon.to_dict())
exp = RamseyExperiment(qubit=q) # 自动用 CONFIG.pulse.t_rabi 等
result = exp.run()
```

## 6.5 向后兼容

```
# 旧 import 仍然可用（通过 src/ 不变 + src_mirror/ 重导出）
from src.qubit import TransmonQubit # 原始 src/（未被修改）
from src_mirror.qubit import TransmonQubit # facade → sqc/
```



```
# 两者均可工作，但新代码建议直接用 sqc/
from sqc.devices.transmon import TransmonQubit
```

## 6.6 运行测试

```
pytest tests/unit -v           # 快速单元测试 (~10s)
pytest tests/regression -v     # 物理回归测试 (~60s)
pytest tests/ -v               # 全部测试 (~15min)
```

## 6.7 启动 Web Demo

```
python web_demo.py           # 旧版 Gradio demo (兼容)
python web_demo_v2.py        # 新版可视化 demo
```

# 7. 扩展指南

本节给出完整的扩展配方，覆盖添加新协议、新算法、新失真模型、新 qubit 类型等场景。

## 7.1 添加新的实验协议

**场景：**实现 ALLXY、T1、CPMG 等新的标准实验，或自定义磁通传感协议。

**步骤：**

1. 新建文件 `sqc/experiments/my_protocol.py`
2. 继承 **Experiment ABC**，实现 `build_sequence()` 和 `run()`：

```
from __future__ import annotations
from dataclasses import dataclass, field
import numpy as np
from qutip import QobjEvo, basis, mesolve

from sqc.config import CONFIG
from sqc.experiments.base import Experiment
from sqc.control.flux_signal import FluxSignal
from sqc.control.sequence import create_my_pulse_sequence
from sqc.simulation.result import ExperimentResult

@dataclass
class MyProtocolExperiment(Experiment):
    """My new sensing protocol.

    Physical model:
        <一句话描述>

    Parameters
    -----
    qubit : TransmonQubit
```

```

flux_signal : FluxSignal or None
param1 : float
    <描述>
"""
qubit: object # TransmonQubit (duck typed)
flux_signal: FluxSignal | None = None
param1: float = 1.0
t_rabi: np.ndarray = field(
    default_factory=lambda: CONFIG.pulse.t_rabi.copy()
)

def __post_init__(self):
    if self.flux_signal is None:
        self.flux_signal = FluxSignal(
            type=3, t_list=CONFIG.pulse.t_signal.copy(),
            amplitude=0.01, center=50, width=5,
        )

def build_sequence(self):
    return create_my_pulse_sequence(self.t_rabi, ...)

def run(self) -> ExperimentResult:
    self.qubit.qubit_in_mag(self.flux_signal)
    # ... mesolve loop ...
    return ExperimentResult(
        data={"p_e": p_e_array, "flux_samples": self.flux_signal.samples.copy()},
        axes={"tau": tau_list, "t_flux": self.flux_signal.t_list.copy()},
        metadata={
            "experiment": "MyProtocolExperiment",
            "qubit_spec": self.qubit.spec() if hasattr(self.qubit, "spec") else None,
        },
        config={"param1": self.param1},
    )

```

### 3. 在 `src_mirror/protocal.py` 添加新的 case 转发 (如需向后兼容):

```

case <new_case_number>:
    from sqc.experiments.my_protocol import MyProtocolExperiment
    exp = MyProtocolExperiment(qubit=qubit, **self.params)
    result = exp.run()
    return (result.data["p_e"], result.axes["tau"]) # 匹配旧 tuple 格式

```

### 4. 添加单元测试 `tests/unit/test_my_protocol.py`:

```

def test_my_protocol_smoke():
    from sqc.experiments.my_protocol import MyProtocolExperiment
    from sqc.devices.transmon import TransmonQubit
    q = TransmonQubit(EC=2*np.pi*0.2, EJ=2*np.pi*15, T1=10000, T2=8000)
    result = MyProtocolExperiment(qubit=q).run()
    assert "p_e" in result.data
    assert len(result.data["p_e"]) > 0

```

### 5. 添加集成测试 `tests/integration/test_my_protocol_experiment.py`:

```
def test_my_protocol_matches_baseline():
    # 类似 test_ramsey_experiment_matches_baseline 的模板
    ...
```

### 6. 生成 **baseline**: 在 `tests/regression/generate_baselines.py` 添加 `_baseline_my_protocol()`, 运行 `python -m tests.regression.generate_baselines` 生成 pickle。

## 7.2 添加新的重建算法

**场景**: 实现新的反演算法 (如 deep learning 反演、压缩感知、Volterra 核展开)。

**步骤**:

1. 新建文件 `sqc/reconstruction/my_algo.py`
2. 继承 **Reconstruction ABC**:

```
from __future__ import annotations
from dataclasses import dataclass
import numpy as np

from sqc.reconstruction.base import Reconstruction
from sqc.simulation.result import ExperimentResult
from sqc.control.flux_signal import FluxSignal

@dataclass
class MyReconstruction(Reconstruction):
    """One-line description.

    Physical model:
        <描述>

    Parameters
    -----
    param : float
    """
    param: float = 1.0

    def reconstruct(self, measurement: ExperimentResult,
                    kernel: np.ndarray | None = None,
                    calibration=None,
                    **kwargs) -> FluxSignal:
        delta_p = measurement.data["delta_p"]
        # ... 算法逻辑 ...
        return FluxSignal(type=8, t_list=t_rec, signal=phi_rec)
```

### 3. 添加 **facade** (如需): 在 `src_mirror/analysis.py` 的 `Analysis` 类中添加方法:

```
def my_method(self, qubit, delta_p, kernel, dt, **kwargs):
    from sqc.reconstruction.my_algo import MyReconstruction
```

```
recon = MyReconstruction(param=kwargs.get("param", 1.0))
meas = ExperimentResult(data={"delta_p": delta_p}, axes={"scan": ...})
return recon.reconstruct(meas, kernel=kernel)
```

4. 添加测试 (unit + integration)，并在 `tests/regression/generate_baselines.py` 中添加对应的 baseline。

## 7.3 添加新的失真模型

**场景：**实现非线性失真、温度依赖失真、串扰耦合等更复杂的模型。

**步骤：**

1. 在 `sqc/hardware/distortion.py` 添加新类：

```
@dataclass
class MyDistortion(DistortionModel):
    """My custom distortion model."""
    param1: float
    param2: float

    def apply(self, waveform: np.ndarray, dt: float) -> np.ndarray:
        # 离散时间卷积或滤波
        ...

    def step_response(self, t: np.ndarray) -> np.ndarray: ...
    def impulse_response(self, t: np.ndarray) -> np.ndarray: ...
    def frequency_response(self, omega: np.ndarray) -> np.ndarray: ...
```

2. 在 `src_mirror/distortion.py` 添加 re-export：

```
from sqc.hardware.distortion import (
    DistortionModel,
    SingleExponentialDistortion,
    MyDistortion,          # 新增
    ...
)
__all__ = ["DistortionModel", ..., "MyDistortion"]
```

3. 添加单元测试 `tests/unit/test_distortion_models.py`：测试 `step_response` 的解析正确性、`apply` 的 DC 增益、`frequency_response` 与时域一致性。

## 7.4 添加新的 FluxSignal 类型

**场景：**实现新的标准磁通波形（如 chirp、伪随机序列、半sine）。

**步骤：**

1. 修改 `sqc/control/flux_signal.py` 的 `_generate()` 函数：

```
@staticmethod
def _generate(type, t_list, params):
```

```

t = np.asarray(t_list, dtype=float)
if type == 0:
    ...
elif type == 9:  # 新类型 9
    amplitude = params.get("amplitude", 1.0)
    f0 = params.get("f0", 0.01)
    chirp_rate = params.get("chirp_rate", 1e-4)
    return amplitude * np.sin(2*np.pi*(f0*t + 0.5*chirp_rate*t**2))
else:
    raise ValueError(...)

```

## 2. 在 `_fill_default_params()` 中添加默认值:

```

@staticmethod
def _fill_default_params(params):
    defaults = {
        "amplitude": 1.0, "frequency": 0.01, ...,
        "chirp_rate": 1e-4,  # 新增
    }
    return {**defaults, **params}

```

## 3. 添加单元测试 `tests/unit/test_flux_signal.py`:

```

def test_flux_signal_chirp():
    t = np.linspace(0, 100, 1000)
    s = FluxSignal(type=9, t_list=t, amplitude=1.0, f0=0.01, chirp_rate=1e-4)
    assert s.samples.shape == t.shape
    # 验证瞬时频率

```

## 7.5 添加新的 qubit 类型（如 Fluxonium）

**场景：**扩展到非 Transmon 量子比特。

**步骤：**

1. 新建文件 `sqc/devices/fluxonium.py`
2. 继承 **Device ABC**:

```

@dataclass(frozen=True)
class FluxoniumSpec:
    name: str
    EC: float
    EJ: float
    EL: float  # inductive energy
    flux_bias: float = 0.0
    n_levels: int = 5  # Fluxonium 通常需要更多能级

    def frequency(self, flux=None) -> float: ...
    def anharmonicity(self) -> float: ...

```

```
class FluxoniumQubit(Device):
    def __init__(self, EC, EJ, EL, ...): ...
    def get_hamiltonian(self) -> Qobj: ...
    def get_hamiltonian_rwa(self, omega_d) -> Qobj: ...
    def get_collapse_operators(self) -> list[Qobj]: ...
```

3. 复用现有 **experiments/calibration/reconstruction**：因为它们都通过 ABC 接口工作，只要 **FluxoniumQubit** 提供 **frequency**, **get\_hamiltonian** 等接口，所有上层模块自动支持。

## 7.6 修改默认时间分辨率（增大 AWG 采样率）

**场景**：研究需要更高时间分辨率的脉冲。

**步骤**：

```
from sqc.config import reconfigure
from sqc.experiments.ramsey import RamseyExperiment

# 使用 4 GSa/s (dt = 0.25 ns)
cfg = reconfigure(sample_rate=4.0, t_rabi_duration=10.0)
exp = RamseyExperiment(qubit=q, t_rabi=cfg.pulse.t_rabi)
# t_rabi 现在有 40 点而非默认 20 点
```

或者全局修改：

```
import sqc.config
sqc.config.CONFIG = sqc.config.Config(
    awg=sqc.config.AWGConfig(sample_rate=4.0),
    pulse=sqc.config.PulseConfig(dt=0.25, t_rabi_duration=10.0),
)
```

**注意**：修改 CONFIG 后，所有 baseline 都需要重新生成。

## 7.7 扩展规范（强制性）

- **命名**：类名用 PascalCase，文件名用 snake\_case。
- **类型注解**：所有公共方法必须有类型注解（`from __future__ import annotations`）。
- **docstring**：英文，numpy 风格，包含 Parameters / Returns / Notes。
- **不可变性偏好**：新数据结构优先使用 `@dataclass(frozen=True)`。
- **facade 保留**：`src_mirror/` 应始终提供向后兼容的重导出。
- **测试要求**：每个新模块  $\geq 3$  个单元测试，涉及物理计算的必须有回归测试。
- **配置来源**：新代码必须从 `CONFIG.pulse.*` 获取时间轴，**禁止 `np.linspace` 硬编码**。
- **依赖方向**：严格遵守 §3.5 依赖矩阵，禁止反向依赖。

## 7.8 添加新 qubit 参数必须通过 sanity check

引入新参数前验证其落在 Gao 2021 推荐范围内 (§2.5)，偏离范围须在 commit message 中说明物理动机。

## 7.9 R1 硬约束：永远不修改 src/

- Track A（重构）**永远不修改、不删除、不重命名 `src/` 下的任何文件**。

- 任何重构相关 PR 中，`git diff master -- src/` 必须为空。
- 所有 facade、wrapper、镜像导出代码，一律放在 `src_mirror/` 目录下。

## 8. API 参考

### 8.1 核心类索引

| 类                           | 模块                           | 用途                 |
|-----------------------------|------------------------------|--------------------|
| Config                      | sqc.config                   | 全局配置聚合             |
| CONFIG                      | sqc.config                   | 全局单例               |
| reconfigure()               | sqc.config                   | 创建自定义 Config       |
| QubitSpec                   | sqc.devices.transmon         | 不可变 qubit 参数       |
| TransmonQubit               | sqc.devices.transmon         | 向后兼容 qubit 类       |
| Resonator                   | sqc.devices.resonator        | 多模谐振腔              |
| ChipTopology                | sqc.devices.chip             | 多 qubit 芯片拓扑       |
| CoupledSystem               | sqc.devices.chip             | 兼容双 qubit + cavity |
| Waveform                    | sqc.control.waveform         | 通用时域波形             |
| CompositeWaveform           | sqc.control.waveform         | 拼接波形               |
| FluxSignal                  | sqc.control.flux_signal      | 磁通信号               |
| Pulse                       | sqc.control.pulse            | 微波控制脉冲             |
| CompositePulse              | sqc.control.pulse            | 复合脉冲序列             |
| ControlLine                 | sqc.hardware.control_line    | 物理控制线              |
| DistortionModel             | sqc.hardware.distortion      | 失真模型 ABC           |
| SingleExponentialDistortion | sqc.hardware.distortion      | 单指数失真              |
| MultiExponentialDistortion  | sqc.hardware.distortion      | 多指数失真              |
| FIRDistortion               | sqc.hardware.distortion      | FIR 滤波失真           |
| IIRDistortion               | sqc.hardware.distortion      | IIR 滤波失真           |
| CustomTransferDistortion    | sqc.hardware.distortion      | 自定义 $H(\omega)$    |
| TransferMatrix              | sqc.hardware.transfer_matrix | Z 线串扰矩阵            |
| IdealProjectiveReadout      | sqc.hardware.readout         | 投影读出               |
| IQReadoutModel              | sqc.hardware.readout         | IQ 解调读出            |
| HamiltonianBuilder          | sqc.simulation.hamiltonian   | 无副作用 H 构造          |
| MesolveRunner               | sqc.simulation.runner        | mesolve 执行器        |
| SlidingMeasurementRunner    | sqc.simulation.runner        | 滑动测量               |
| ExperimentResult            | sqc.simulation.result        | 实验结果容器             |

| 类                               | 模块                                     | 用途               |
|---------------------------------|----------------------------------------|------------------|
| Experiment                      | sqc.experiments.base                   | 实验 ABC           |
| RabiExperiment                  | sqc.experiments.rabi                   | Rabi 振荡          |
| RamseyExperiment                | sqc.experiments.ramsey                 | Ramsey 协议        |
| DiffEchoExperiment              | sqc.experiments.echo                   | 差分回波             |
| TransientSensingExperiment      | sqc.experiments.transient              | 瞬态传感             |
| CryoscopeExperiment             | sqc.experiments.cryoscope              | Cryoscope        |
| Reconstruction                  | sqc.reconstruction.base                | 重建 ABC           |
| KernelEstimator                 | sqc.reconstruction.kernel              | 控制核估计            |
| WienerReconstruction            | sqc.reconstruction.wiener              | Wiener 反卷积       |
| RamseyIQReconstruction          | sqc.reconstruction.wiener              | Ramsey IQ 反演     |
| RamseyUnwrapReconstruction      | sqc.reconstruction.wiener              | Ramsey 解缠绕       |
| DiffEchoReconstruction          | sqc.reconstruction.wiener              | 差分回波反演           |
| HammersteinWienerReconstruction | sqc.reconstruction.hammerstein         | 非线性反演            |
| LMReconstruction                | sqc.reconstruction.numerical_inverse   | LM 数值反演          |
| CryoscopeReconstruction         | sqc.reconstruction.cryoscope           | Cryoscope 反演     |
| Calibration                     | sqc.calibration.base                   | 标定 ABC           |
| CalibrationTable                | sqc.calibration.base                   | 标定结果             |
| QubitFrequencyCalibration       | sqc.calibration.qubit_frequency        | f01 标定           |
| FluxResponseCalibration         | sqc.calibration.flux_response          | f( $\Phi$ ) 标定   |
| TransferFunctionCalibration     | sqc.calibration.transfer_function      | H( $\omega$ ) 标定 |
| PredistortionDesigner           | sqc.calibration.predistortion          | 预失真设计器           |
| Workflow                        | sqc.workflows.base                     | 顶层流程 ABC         |
| PredistortionValidationWorkflow | sqc.workflows.predistortion_validation | 预失真验证            |
| ZCrosstalkWorkflow              | sqc.workflows.z_crosstalk              | Z 串扰提取           |

8.2 关键函数签名

```
# Config
from sqc.config import CONFIG, reconfigure
CONFIG.awg.dt                # 0.5 ns
CONFIG.pulse.t_rabi          # np.ndarray
CONFIG.pulse.t_global        # np.ndarray
CONFIG.transmon.to_dict()    # dict for TransmonQubit
reconfigure(sample_rate=4.0, t_rabi_duration=20) -> Config

# HamiltonianBuilder
HamiltonianBuilder.build(qubit, flux_signal, pulse, frame, omega_d)
```



```

-> tuple[list, np.ndarray]

# KernelEstimator
KernelEstimator(stim_amplitude=0.0215, stim_width=3.0).estimate(pulse, qubit)
-> tuple[np.ndarray, np.ndarray] # (t_samples, kernel)

# WienerReconstruction
WienerReconstruction(lambda_reg=1.0).reconstruct(
    measurement, kernel, calibration=None, dt=None
) -> FluxSignal

# LMReconstruction
LMReconstruction(
    qubit, control_pulse, basis_type="fourier", n_basis=100,
    lambda_reg=100.0, max_iter=10, tol=1e-6, mu_init=1e-3,
    use_adjoint=True,
).reconstruct(measurement, initial_guess=None) -> tuple[FluxSignal, dict]

# ControlLine
ControlLine.apply(awg_waveform: Waveform) -> Waveform
ControlLine.predistort(target_waveform: Waveform, designer: PredistortionDesigner) ->
Waveform

# TransferMatrix
TransferMatrix.apply(source_voltages: dict[str, Waveform]) -> dict[str, FluxSignal]
TransferMatrix.from_dc_matrix(dc_matrix, source_names, target_names) -> TransferMatrix

# PredistortionDesigner
PredistortionDesigner(method="fir_inverse", n_taps=64, regularization=1e-4)
    .design(transfer_model, dt) -> DistortionModel
    .predistort(target, transfer) -> Waveform

# DistortionModel (ABC)
.apply(waveform, dt) -> np.ndarray
.apply_to_waveform(wf) -> Waveform
.step_response(t) -> np.ndarray
.impulse_response(t) -> np.ndarray
.frequency_response(omega) -> np.ndarray # complex

# ReadoutModel
IdealProjectiveReadout().measure(state, qubit=None) -> dict[str, float]
IQReadoutModel(tau, t_rabi, omega_d).measure(qubit, **extra) -> dict[str, float]

```

### 8.3 Cheat Sheet (最常用 5 行)

```

from sqc.config import CONFIG
from sqc.devices.transmon import TransmonQubit
from sqc.experiments.ramsey import RamseyExperiment

q = TransmonQubit(**CONFIG.transmon.to_dict())
result = RamseyExperiment(qubit=q).run()
print(result.data["p_e"]) # done

```

## 9. 测试与回归

### 9.1 测试套件

| 套件    | 位置                 | marker                   | 用途          |
|-------|--------------------|--------------------------|-------------|
| 单元测试  | tests/unit/        | @pytest.mark.unit        | 纯函数/类单元验证   |
| 集成测试  | tests/integration/ | @pytest.mark.integration | 跨模块管道验证     |
| 回归测试  | tests/regression/  | @pytest.mark.regression  | 物理结果锚定      |
| 等价性测试 | tests/equivalence/ | —                        | 新 sqc 代码自治性 |

### 9.2 当前测试统计 (v2.0)

```
228 passed
├─ unit: 185
├─ equivalence: 12 (含 case5)
├─ integration: 24
└─ regression: 7
```

### 9.3 物理回归机制

1. **baseline 生成**: `python -m tests.regression.generate_baselines`
2. **回归验证**: `pytest tests/regression -m regression`
3. **容限**: `rtol=1e-6, atol=1e-9` (**不可放宽**——若数值偏移须调查根因)
4. **baseline 更新**: 仅当有意改变物理行为时重生成, commit message 必须显式说明

### 9.4 baseline 清单

| 文件                        | 内容                                  | 生成自                     |
|---------------------------|-------------------------------------|-------------------------|
| qubit_static.pkl          | $f_{01}$ , $\alpha$ , $\kappa$ 静态属性 | _baseline_qubit_static  |
| ramsey_default.pkl        | RamseyExperiment (sqc, arange 时间轴)  | _baseline_ramsey        |
| diff_echo_default.pkl     | DiffEchoExperiment                  | _baseline_diff_echo     |
| transient_default.pkl     | TransientSensingExperiment          | _baseline_transient     |
| lm_default.pkl            | LM 数值反演                             | _baseline_lm            |
| predistortion_default.pkl | 预失真验证                               | _baseline_predistortion |
| z_crosstalk_default.pkl   | Z 串扰提取                              | _baseline_z_crosstalk   |

### 9.5 CI 守则

- 每次 PR 必须跑 `pytest tests/ -v`, 全绿才能 merge。
- 任何修改 `src/*.py` 的 PR 会被自动拒绝 (CI 检查 `git diff master -- src/` 必须为空)。
- 任何放宽 `rtol/atol` 容限的 PR 须 reviewer 批准并附物理动机。
- baseline 改变须在 commit message 中显式说明 ("intentional physics change: ...")。

## 10. 全局配置 (Global Configuration)

## 10.1 设计理念

`sqc/config.py` 提供统一的全局配置体系。参数从**硬件层向上推导**，而不是在不同文件中各自硬编码。

```
AWG sample_rate (hardware)
→ dt = 1 / sample_rate (全局时间量子)
→ t_rabi = arange(0, 10, dt) (标准  $\pi$  脉冲窗口)
→ t_global = arange(-50, 400, dt) (仿真时间窗)
→ t_signal = arange(0, 250, dt) (磁通信号默认时间轴)
```

所有时间轴使用 `np.arange(start, stop, dt)` 而非 `np.linspace(start, stop, N)`，以确保：

- 每个时间点落在整数 AWG 采样边界上
- `dt` 全局唯一，从 `AWGConfig.sample_rate` 推导
- 脉冲面积计算不依赖不整齐的步长 (`amplitude = pi / duration`)

## 10.2 配置层次

| Section        | Dataclass                         | 关键字段                                                                   |
|----------------|-----------------------------------|------------------------------------------------------------------------|
| Hardware       | <code>AWGConfig</code>            | <code>sample_rate: 2.0 GSa/s → dt: 0.5 ns</code>                       |
|                | <code>ControlLineDefaults</code>  | <code>impedance: 50 <math>\Omega</math>, attenuation_db: 20 dB</code>  |
| Devices        | <code>TransmonDefaults</code>     | <code>EC: 0.2, EJ: 15.0, T1: 10k ns, n_levels: 3</code>                |
| Control        | <code>PulseConfig</code>          | <code>t_rabi_duration: 10 ns, t_global_start/end, dt (injected)</code> |
| Simulation     | <code>SimulationConfig</code>     | <code>atol: 1e-8, rtol: 1e-6</code>                                    |
| Reconstruction | <code>ReconstructionConfig</code> | <code>lambda_reg: 1.0, stim_amplitude: 0.0215, lm_n_basis: 100</code>  |

## 10.3 使用方式

```
from sqc.config import CONFIG, reconfigure

# 获取全局默认值
dt = CONFIG.awg.dt                # 0.5 ns
t_rabi = CONFIG.pulse.t_rabi      # arange(0, 10, dt)
t_cons = CONFIG.transmon          # TransmonDefaults
q = TransmonQubit(**t_cons.to_dict())

# 创建自定义配置（全局单例不变）
cfg2 = reconfigure(sample_rate=4.0, t_rabi_duration=20)
exp = RamseyExperiment(qubit=q, t_rabi=cfg2.pulse.t_rabi)
```

## 10.4 已适配模块

所有时间轴从 `CONFIG.pulse` 获取默认值：

- `sqc/experiments/`: Ramsey, Echo, Rabi, Transient, Cryoscope
- `sqc/calibration/`: QubitFrequency, FluxResponse
- `sqc/control/`: sequence.py (free-evolution gaps), flux\_signal.py (default `t_signal`)

- `sqc/hardware/`: `readout.py` (`t_rabi`)
- `sqc/workflows/`: `z_crosstalk.py` (`t_rabi`)

## 10.5 配置 API 详解

### AWGConfig

```
@dataclass(frozen=True)
class AWGConfig:
    sample_rate: float = 2.0      # GSa/s
    voltage_range: float = 2.0    # Vpp
    resolution: int = 14          # bits

    @property
    def dt(self) -> float:
        return 1.0 / self.sample_rate # ns
```

### PulseConfig

```
@dataclass(frozen=True)
class PulseConfig:
    dt: float                                # from AWGConfig.dt
    t_rabi_duration: float = 10.0
    t_pi2_duration: float = 10.0
    t_global_start: float = -50.0
    t_global_end: float = 400.0
    t_signal_duration: float = 250.0
    t_signal_start: float = 0.0
    tau_stride: int = 1

    @property
    def t_rabi(self) -> np.ndarray: ...
    @property
    def t_global(self) -> np.ndarray: ...
    @property
    def t_signal(self) -> np.ndarray: ...
    @property
    def tau_list(self) -> np.ndarray: ...

    def make_time(self, t_start, t_end) -> np.ndarray:
        return np.arange(t_start, t_end, self.dt)
```

### TransmonDefaults.to\_dict()

返回适合 `TransmonQubit(**kwargs)` 构造的 dict (含  $2\pi$  单位换算)。

## 10.6 内部约定：私有标识符以 `_` 开头

`_GT`, `_gap()` 等模块内部使用的常量/辅助函数以下划线开头——这是 Python 标准约定：单下划线前缀表示“模块私有，外部不应直接 `import`”。例如：

```
# sqc/control/sequence.py
_GT = CONFIG.awg.dt          # 仅在 sequence.py 内部使用
def _gap(duration): ...      # 仅在 sequence.py 内部使用
```

外部代码应通过 `CONFIG.awg.dt` 或 `CONFIG.pulse.make_time(...)` 获取等价功能，不要 `import _GT` 或 `_gap`。

## 11. 设计原则与约定

### 11.1 核心原则

- 1. **物理结果不变**：重构不改变任何物理算法的等价行为
- 2. **device/qubit 只描述物理参数**：不存储实验状态
- 3. **HamiltonianBuilder 无副作用**：输入不被修改
- 4. **experiment 是组合**：不是巨函数
- 5. **reconstruction 只消费数据**：不做仿真（LM 通过依赖注入）
- 6. **hardware 显式建模**：反映真实 cQED 栈
- 7. **永久镜像**：`src/` 不删除，旧代码无限期可工作
- 8. **不引入新依赖**：只用 `numpy/scipy/qutip/matplotlib/dataclasses`
- 9. **全局配置中心化**：所有时间网格从 `CONFIG.awg.dt` 推导，不使用 `np.linspace`
- 10. **类型注解优先**：所有新代码使用 `from __future__ import annotations`
- 11. **不引入配置文件**：所有参数通过 Python 对象传递，不用 `yaml/toml`

### 11.2 命名与拼写

- `Protocal`（故意错拼）在 `src/` 和 `src_mirror/` 中**永远保留**
- `sliding_measrement`（故意错拼）在 `facade` 中保留
- `sqc/` 中新代码使用正确拼写：`Protocol`, `Experiment`, `Calibration`
- 类名用 `PascalCase`（`RamseyExperiment`、`WienerReconstruction`）
- 文件名用 `snake_case`（`ramsey.py`、`wiener.py`）
- 私有标识符以 `_` 开头（`_GT`、`_gap`）

### 11.3 单位约定

| 物理量     | 单位                 | 说明                                                     |
|---------|--------------------|--------------------------------------------------------|
| 频率/能量   | rad-GHz（含 $2\pi$ ） | 代码中 <code>EC=2*pi*0.2</code> 即 $EC/h = 200\text{ MHz}$ |
| 时间      | ns                 | 与 QuTiP 默认一致                                           |
| 磁通      | $\Phi_0$           | 量子磁通                                                   |
| 角度      | rad                | $\pi=\pi$ ，不用 <code>degree</code>                      |
| $\hbar$ | 1                  | 自然单位                                                   |

### 11.4 Hamiltonian list 格式（QuTiP 兼容）

```
H_list = [
    H_static,          # Qobj
    [H_op_1, coeff_array_1], # 时变项 1（数组形式）
    [H_op_2, "sin(w*t)"],   # 时变项 2（字符串形式，备用）
]
```

```
...
]
QobjEvo(H_list, tlist=t_array)
mesolve(H_list, psi0, t_array, c_ops, e_ops)
```

11.5 与 Gao 2021 的差异

| 项         | 论文                                              | 本项目                     | 理由             |
|-----------|-------------------------------------------------|-------------------------|----------------|
| 算符表示      | ladder operator                                 | QuTiP destroy/num       | 数值等价           |
| 噪声模型      | 多渠道 $T_1, T_2, T_\phi$                          | Lindblad c_ops          | 标准 QuTiP, 易扩展  |
| Readout   | dispersive cavity + IQ                          | Ramsey-based IQ         | 本项目核心是磁通传感     |
| 校准循环      | 完整 Fig.9 依赖图                                    | 简化为三类                   | 聚焦三大主线         |
| 双比特门      | flux-pulsing + microwave + parametric           | 仅 flux-pulsing iSWAP/CZ | 项目当前单 qubit 为主 |
| Cavity 表征 | $\S V.F$ 完整 (number splitting, revival, Wigner) | 不实现                     | 不在主线           |

11.6 R1 硬约束（最高优先级）

- Track A 重构**永远不修改、不删除、不重命名** `src/` 下的任何文件
- 任何重构 PR 中 `git diff master -- src/` 必须为空
- 所有 facade/wrapper/mirror 代码**一律放在** `src_mirror/` 目录下
- 这条规则由 CI 自动检查，违反者 PR 直接拒绝

12. 附录 — Gao 2021 公式对应表

本节按 Gao 2021 章节顺序，列出每个公式/章节在 `src/` 中的对应代码。便于读论文时快速定位实现。

12.1 §II.B–C Transmon 物理

| Gao 公式    | 含义                                                 | src 对应                                               |
|-----------|----------------------------------------------------|------------------------------------------------------|
| Eq. 13    | $H = 4 E_C n^2 - E_J \cos(\varphi)$                | <code>TransmonQubit.get_hamiltonian()</code>         |
| Eq. 15–17 | Duffing 振子展开                                       | 同上（内部展开）                                             |
| Eq. 18    | $\omega_T = \sqrt{8 E_J E_C} - E_C, \alpha = -E_C$ | <code>QubitSpec.frequency(), .anharmonicity()</code> |
| Eq. 20    | $E_J(\Phi) = E_J^0  \cos(\pi \Phi / \Phi_0) $      | <code>QubitSpec.EJ_at(flux)</code>                   |

12.2 §II.A 谐振腔

| Gao 公式  | 含义                 | src 对应                                               |
|---------|--------------------|------------------------------------------------------|
| Eq. 2–7 | LC 谐振腔 Hamiltonian | <code>Resonator.get_hamiltonian()</code>             |
| Eq. 8   | 输入输出关系             | <code>DistortionModel.apply()</code> (本项目泛化为 LTI 滤波) |

12.3 §II.E 色散耦合

| Gao 公式 | 含义                                              | sqc 对应                                       |
|--------|-------------------------------------------------|----------------------------------------------|
| Eq. 33 | Qubit-cavity 色散耦合 H                             | <code>CoupledSystem.get_hamiltonian()</code> |
| Eq. 35 | $\chi = 2 g^2 \alpha / (\Delta(\Delta+\alpha))$ | 隐含在 <code>CoupledSystem</code> 参数化           |

12.4 §III.B 参数推荐范围

| Gao 表 | 参数                            | sqc 默认值                                |
|-------|-------------------------------|----------------------------------------|
| 表 I   | $E_J/h \in [10, 25]$ GHz      | <code>CONFIG.transmon.EJ = 15.0</code> |
| 表 I   | $E_C/h \in [160, 400]$ MHz    | <code>CONFIG.transmon.EC = 0.2</code>  |
| 表 I   | $E_J/E_C \approx 50$          | 默认 ~75                                 |
| 表 I   | $f_{01} \in [4, 8]$ GHz       | 默认 ~4.7 GHz                            |
| 表 I   | $\alpha/h \in [200, 300]$ MHz | 默认 200 MHz                             |

12.5 §IV.B 信号处理

| Gao 公式 | 含义                                                            | sqc 对应                                     |
|--------|---------------------------------------------------------------|--------------------------------------------|
| Eq. 10 | $H_d = \epsilon(t) a^\dagger + \epsilon^*(t) a$               | <code>Pulse.get_hamiltonian()</code>       |
| Eq. 37 | $\Gamma_D \approx \omega_q^2 Z_0 C_c^2 / C_\Sigma$<br>Purcell | (项目未直接实现 Purcell)                          |
| Eq. 49 | $\Theta(t) = -\Omega V_0 / \hbar \int_0^t s(t') dt'$          | <code>Pulse.get_angle_simple()</code>      |
| Eq. 52 | DRAG 脉冲                                                       | <code>TransmonQubit.simulate_gate()</code> |

12.6 §V.B 单比特实验

| Gao 公式        | 含义                | sqc 对应                                          |
|---------------|-------------------|-------------------------------------------------|
| Eq. 53        | $T_1$ 衰减          | (用 <code>RamseyExperiment</code> 改 sequence 实现) |
| Eq. 54        | Ramsey $T_2^*$    | <code>RamseyExperiment</code>                   |
| Eq. 55        | Hahn echo $T_2^E$ | <code>create_echo_pulse</code>                  |
| Fig. 11 ALLXY | DRAG 校准           | (可作 P5+ 扩展, 详见 phase_2_handbook §3.8)           |

12.7 §V.C 读出

| Gao 公式   | 含义             | sqc 对应                                |
|----------|----------------|---------------------------------------|
| Eq. 60   | Fidelity F     | (作为 P5+ 扩展, 见 phase_3_handbook §3.10) |
| Eq. 61   | QND-ness Q     | 同上                                    |
| Eq. 62   | $\Lambda_M$ 矩阵 | 同上                                    |
| Fig. 13a | Butterfly 实验   | 同上                                    |

12.8 §V.D 双比特门

| Gao 公式    | 含义                 | sqc 对应                                                              |
|-----------|--------------------|---------------------------------------------------------------------|
| Eq. 75    | $\zeta_{ij}$ 残余 ZZ | (P5 ZCrosstalkWorkflow 间接测)                                         |
| Eq. 75–85 | Z-crosstalk 完整理论   | <a href="#">TransferMatrix</a> + <a href="#">ZCrosstalkWorkflow</a> |

12.9 §V.E Cryoscope

| Gao 概念         | 含义               | sqc 对应                                                                                   |
|----------------|------------------|------------------------------------------------------------------------------------------|
| Cryoscope 协议   | 截断 + IQ 测相位      | <a href="#">CryoscopeExperiment</a>                                                      |
| 失真模型 (IIR fit) | 多指数尾巴拟合          | <a href="#">MultiExponentialDistortion</a> + <a href="#">TransferFunctionCalibration</a> |
| 预失真设计          | 逆滤波器             | <a href="#">PredistortionDesigner</a>                                                    |
| Z-crosstalk 矩阵 | $H_{ij}(\omega)$ | <a href="#">TransferMatrix</a>                                                           |

12.10 §V.F Cavity 表征（本项目不在主线）

| Gao 公式   | 含义                 | sqc 对应                                          |
|----------|--------------------|-------------------------------------------------|
| Fig. 17b | Number splitting   | (P5+, 见 <a href="#">phase_5_handbook §3.5</a> ) |
| Eq. 87   | Ramsey revival     | 同上                                              |
| Eq. 89   | Wigner $W(\alpha)$ | 同上                                              |

参考文献

- **[Gao 2021]** Y. Y. Gao, M. A. Rol, S. Touzard, and C. Wang, "Practical Guide for Building Superconducting Quantum Devices", *PRX Quantum* **2**, 040202 (2021). DOI: 10.1103/PRXQuantum.2.040202. **本项目架构的主要参考。**
- **[Koch 2007]** J. Koch et al., "Charge-insensitive qubit design derived from the Cooper pair box", *Phys. Rev. A* **76**, 042319 (2007). Transmon 原始论文。
- **[Motzoi 2009]** F. Motzoi et al., "Simple Pulses for Elimination of Leakage in Weakly Nonlinear Qubits", *Phys. Rev. Lett.* **103**, 110501 (2009). DRAG 脉冲。
- **[Reed 2010]** M. D. Reed et al., "High-Fidelity Readout in Circuit Quantum Electrodynamics Using the Jaynes-Cummings Nonlinearity", *Phys. Rev. Lett.* **105**, 173601 (2010). 高功率读出。
- **[Rol 2020]** M. A. Rol et al., "Time-domain characterization and correction of on-chip distortion of control pulses in a quantum processor", *Appl. Phys. Lett.* **116**, 054001 (2020). Cryoscope 原始文献。

文档维护

| 版本   | 日期         | 主要变更                                                                         |
|------|------------|------------------------------------------------------------------------------|
| v1.0 | 2026-05-01 | 初版，覆盖六层架构与基本扩展指南                                                             |
| v2.0 | 2026-05-12 | 引入全局配置 (§10)；按 Gao 2021 章节重组 (§12 附录)；每个模块扩展物理对应与扩展点；新增 R1 硬约束说明；测试统计更新到 228 |

下一步阅读：



- 完整设计背景: [idea/refactor/\\_refactor\\_plan.md](#)
  - Phase 工作记录: [idea/refactor/\\_handoff\\_state.md](#)
  - 实战 demo: [python web\\_demo\\_v2.py](#)
- 

文档结束。